Recursos específicos do PostgreSQL que não devem ser usados no projeto

1. Escopo e critério adotado

Este documento serve como material orientativo para garantir a conformidade nas entregas do projeto MATA60. No texto do projeto, o enunciado inclui a regra 6, determinando que "*discentes devem empregar SQL portável conforme o padrão ISO/IEC 9075 (frequentemente chamado de "SQL ANSI") para operar no banco de dados*". Sem fazer uma visita exaustiva do do padrão ISO/IEC 9075, e considerando que nossa disciplina usa o PostgreSQL, para facilitar a consulta da turma e assegurar corretude da entrega, defino SQL ANSI como:

> **SQL ANSI** = Conjunto de recursos que não fazem parte do SQL padronizado. Para o PostgreSQL, trataremos os comandos e cláusulas fora do padrão SQL:2023.

2. DDL: tipos de dados e definição do esquema

2.1. Tipos de dados que não devem ser usados

Considere usar BIGINT, BIT, BIT VARYING, BOOLEAN, CHAR/CHARACTER, VARCHAR/CHARACTER VARYING, DATE, TIME, TIMESTAMP, INTERVAL, SMALLINT, INTEGER, REAL, DOUBLE PRECISION, NUMERIC/DECIMAL e XML. Use os nomes

padronizados em vez de aliases internos como int4, int8, float8, bool ou varbit. No Quadro 1, tento sistematizar comandos usados em entregas anteriores que não são permitidos.

Quadro 1. comandos não permitidos, fora do SQL ANSI

NÃO USAR	Exemplo	Alternativa / orientação
smallserial, serial, bigserial	id SERIAL	INTEGER GENERATED BY DEFAULT AS IDENTITY. Os "serial" são atalhos específicos do PostgreSQL.
text	descricao TEXT	Preferir VARCHAR(n) / CHARACTER VARYING(n) com tamanho definido no modelo.
bytea	arquivo BYTEA	Evitar dados binários no projeto; o tipo BYTEA é próprio do PostgreSQL.
money	preco MONEY	Usar DECIMAL(p,s) ou NUMERIC(p,s).
uuid	id UUID	Se necessário ao domínio, representar com CHAR(36)/VARCHAR(36) no projeto portátil.
json e jsonb	dados JSONB	Modelar relacionalmente. JSONB e seus operadores são PostgreSQL; a lista de tipos SQL padronizados da documentação do PostgreSQL não inclui JSON/JSONB.
jsonpath	filtro JSONPATH	Não usar; é um tipo específico do PostgreSQL para representar expressões SQL/JSON path.
Tipos de rede	ip INET; rede CIDR; MACADDR/MACADDR8	Usar CHAR/VARCHAR quando uma representação textual for suficiente.
Tipos geométricos	POINT, LINE, LSEG, BOX, PATH, POLYGON, CIRCLE	Representar atributos geométricos em colunas numéricas/relacionais conforme o modelo do projeto.
Busca textual	TSVECTOR, TSQUERY	Não usar mecanismos de full-text search nativos do PostgreSQL.

NÃO USAR	Exemplo	Alternativa / orientação
Ranges e multiranges	DATERANGE, TSRANGE, INT4RANGE, ...	Usar colunas de início/fim e restrições SQL padronizadas.
Tipos internos/sistema	OID, REGCLASS, REGPROC, REGTYPE, PG_LSN, PG_SNAPSHOT, ...	Não modelar dados da aplicação com tipos do catálogo interno do PostgreSQL.
ENUM do PostgreSQL	CREATE TYPE status AS ENUM (...)	Usar VARCHAR/CHAR + CHECK, ou uma tabela de domínio referenciada por FK.

2.2. Comandos DDL e objetos específicos do PostgreSQL

Considere usar CREATE / ALTER / DROP TABLE, INSERT, etc. Aqui é importante iniciar com algumas exceções (as equipes podem considerar usar estas, se cabível):

- CREATE / ALTER / DROP DATABASE
- CREATE / ALTER / DROP INDEX; REINDEX
- CREATE / ALTER / DROP MATERIALIZED VIEW; REFRESH MATERIALIZED VIEW
- CREATE / ALTER / DROP RULE
- CREATE / ALTER / DROP EVENT TRIGGER

No Quadro 2, portanto, incluo os comandos e cláusulas proibidas que já foram usados em entregas anteriores.

Quadro 2. comandos DDL não permitidos, fora do SQL ANSI

NÃO USAR	Exemplo	Por quê / alternativa
CREATE TYPE ... AS ENUM / AS RANGE / tipo-base	CREATE TYPE cor AS ENUM (...);	A forma composta de CREATE TYPE pode ser padronizada; as formas ENUM, RANGE, base e shell do PostgreSQL não.

NÃO USAR	Exemplo	Por quê / alternativa
IF NOT EXISTS / IF EXISTS em formas marcadas pela documentação como extensão	CREATE TABLE IF NOT EXISTS t (...); DROP TABLE IF EXISTS t;	Para scripts avaliados estritamente por portabilidade, não usar essas variantes.
CREATE / ALTER / DROP TABLESPACE	CREATE TABLESPACE ...	Tablespaces são recurso físico do PostgreSQL, fora do SQL padronizado.
CREATE / ALTER / DROP EXTENSION	CREATE EXTENSION pg_trgm;	Não usar extensões instaláveis nem objetos dependentes delas.
CREATE / ALTER / DROP MATERIALIZED VIEW; REFRESH MATERIALIZED VIEW	CREATE MATERIALIZED VIEW mv AS ...;	Usar VIEW comum quando suficiente ou consulta SQL; a forma materializada do PostgreSQL é extensão.
CREATE / ALTER / DROP RULE	CREATE RULE ...	O sistema de rewrite rules é extensão específica do PostgreSQL.
CREATE / ALTER / DROP EVENT TRIGGER	CREATE EVENT TRIGGER ...	Event triggers de DDL não existem no padrão SQL.
CREATE / ALTER / DROP POLICY	CREATE POLICY ...	Row-Level Security com CREATE POLICY é extensão do PostgreSQL.
CREATE TYPE ... AS ENUM / AS RANGE / tipo-base	CREATE TYPE cor AS ENUM (...);	A forma composta de CREATE TYPE pode ser padronizada; as formas ENUM, RANGE, base e shell do PostgreSQL não.
Objetos avançados do mecanismo PostgreSQL	CREATE ACCESS METHOD, CREATE OPERATOR CLASS/FAMILY, CREATE TEXT SEARCH ..., CREATE PUBLICATION/SUBSCRIPTION	Não devem compor o projeto: dependem da arquitetura interna, indexação, busca textual ou replicação do PostgreSQL.

3. DML: manipulação e consulta de dados

As exceções aqui estão limitadas a SELECT DISTINCT ON (...), LIMIT e ILIKE / NOT ILIKE. O Quadro 3 sistematiza os comandos frequentes que não são permitidos.

Quadro 3. comandos DDL não permitidos, fora do SQL ANSI

NÃO USAR	Exemplo	Alternativa / observação
COPY	COPY aluno FROM ... CSV HEADER;	Carga/descarga em massa específica do PostgreSQL. Para a entrega, usar INSERT ou mecanismo externo não incluído no SQL avaliado.
INSERT ... ON CONFLICT	INSERT ... ON CONFLICT DO UPDATE ...	UPSERT PostgreSQL. Quando necessário e permitido, usar MERGE, que possui forma padronizada.
RETURNING em INSERT/UPDATE/DELETE	INSERT ... RETURNING id;	RETURNING é extensão PostgreSQL.
UPDATE ... FROM	UPDATE a SET x=b.x FROM b WHERE ...;	FROM no UPDATE é extensão. Reescrever com subconsulta correlacionada/EXISTS conforme o caso.
DELETE ... USING	DELETE FROM a USING b WHERE ...;	USING no DELETE é extensão. Usar subconsulta em WHERE/EXISTS.
WITH contendo DML	WITH x AS (DELETE ... RETURNING ...) SELECT ...;	PostgreSQL permite INSERT/UPDATE/DELETE/MERGE dentro de CTE; isso não está no padrão.
MATERIALIZED / NOT MATERIALIZED em CTE	WITH x AS MATERIALIZED (...) ...	Opções de materialização da CTE são extensões PostgreSQL.
SELECT sem FROM	SELECT 2+2;	A documentação PostgreSQL classifica a omissão de FROM como extensão. Em uma entrega estrita, evite essa forma.

NÃO USAR	Exemplo	Alternativa / observação
Locking clauses específicas	FOR NO KEY UPDATE; FOR SHARE; FOR KEY SHARE; NOWAIT; SKIP LOCKED	Essas variantes/opções não estão no padrão. O uso PostgreSQL de FOR UPDATE em SELECT comum também amplia o uso padronizado.
ONLY e "tabela*" com herança	SELECT * FROM ONLY pai;	Semântica ligada à herança de tabelas PostgreSQL.
SELECT INTO para criar tabela	SELECT * INTO nova FROM antiga;	O SQL padrão usa SELECT INTO para variáveis de host; para criação de tabela, usar CREATE TABLE ... AS (...) quando aplicável.
Operadores regex	~ ~* !~ !~*	São operadores PostgreSQL. Não confundir com SIMILAR TO, que tem origem no padrão SQL.
Operadores JSON/JSONB	->, ->>, #>, #>>, @>, ?, ?|, ?& ...	Dependem de tipos e operadores PostgreSQL.
Operadores de arrays PostgreSQL	@>, <@, && e usos específicos de ||	Evitar. Em modelagem relacional, representar múltiplos valores em outra tabela.
Full-text search	@@, to_tsvector(...), to_tsquery(...), plainto_tsquery(...)	Mecanismo de busca textual nativo do PostgreSQL.
Funções de sequência PostgreSQL	nextval(...), currval(...), setval(...)	Preferir identidade padronizada para geração automática de chaves; não depender dessas funções no projeto.
Funções/objetos pg_*	pg_typeof, pg_get_*, pg_advisory_*, pg_notify, etc.	APIs internas/administrativas do PostgreSQL; não usar como parte da lógica SQL do projeto.

4. Referências

- PostgreSQL 18: SQL Conformance: https://www.postgresql.org/docs/18/features.html
- PostgreSQL 18: Data Types: https://www.postgresql.org/docs/18/datatype.html
- PostgreSQL 18: Numeric Types / Serial Types: https://www.postgresql.org/docs/18/datatype-numeric.html
- PostgreSQL 18: CREATE TABLE: https://www.postgresql.org/docs/18/sql-createtable.html
- PostgreSQL 18: ALTER TABLE: https://www.postgresql.org/docs/18/sql-altertable.html
- PostgreSQL 18: SELECT: https://www.postgresql.org/docs/18/sql-select.html
- PostgreSQL 18: INSERT: https://www.postgresql.org/docs/18/sql-insert.html
- PostgreSQL 18: UPDATE: https://www.postgresql.org/docs/18/sql-update.html
- PostgreSQL 18: DELETE: https://www.postgresql.org/docs/18/sql-delete.html
- PostgreSQL 18: CREATE INDEX: https://www.postgresql.org/docs/18/sql-createindex.html
- PostgreSQL 18: CREATE TYPE: https://www.postgresql.org/docs/18/sql-createtype.html
- PostgreSQL 18: CREATE FUNCTION: https://www.postgresql.org/docs/18/sql-createfunction.html
- PostgreSQL 18: CREATE EVENT TRIGGER: https://www.postgresql.org/docs/18/sql-createeventtrigger.html
- PostgreSQL 18: CREATE RULE: https://www.postgresql.org/docs/18/sql-createrule.html
- PostgreSQL 18: CREATE POLICY: https://www.postgresql.org/docs/18/sql-createpolicy.html
- PostgreSQL 18: COPY: https://www.postgresql.org/docs/18/sql-copy.html
- PostgreSQL 18: EXPLAIN: https://www.postgresql.org/docs/18/sql-explain.html
- PostgreSQL 18: VACUUM: https://www.postgresql.org/docs/18/sql-vacuum.html
- PostgreSQL 18: ANALYZE: https://www.postgresql.org/docs/18/sql-analyze.html